LECTURE 07

Networking & APIs

Talking to the internet from your Flutter app

Instructor · Muhammad Sabah

AGENDA

Today's plan

01

What is an API?

Request, response, and JSON

02

GET & POST

Fetch data, send data

03

Parse & display

JSON to Dart objects, then to UI

04

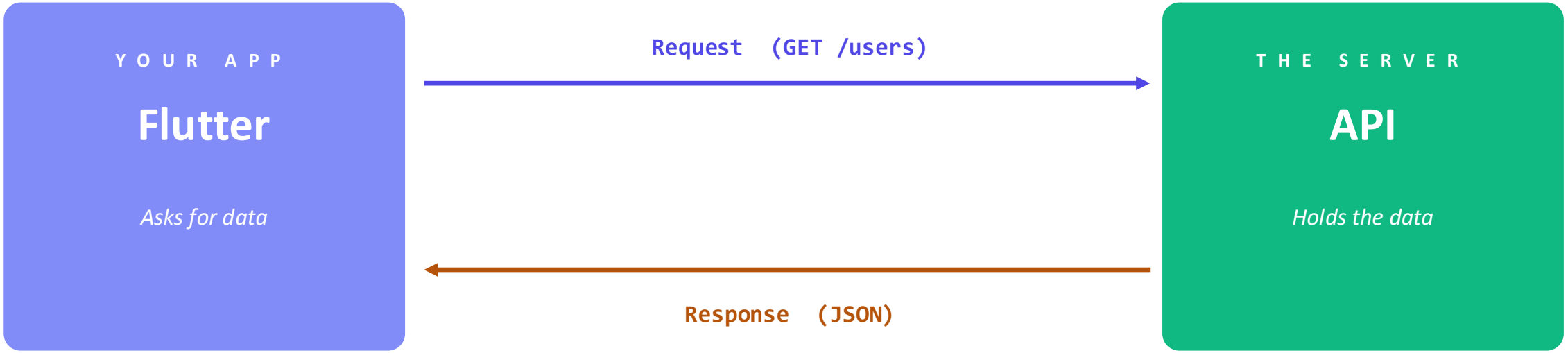
Practice

Fetch real data from a public API

THE BIG IDEA

What is an API?

Your app asks a server for data. The server sends data back. That's it.



JSON looks like:

```
{ "name": "Hama", "age": 25 }
```

The http package

Flutter doesn't ship with networking built in — we use the official http package.

STEP 1 — ADD TO PUBSPEC.YAML

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
```

STEP 2 — RUN

```
flutter pub get
```

STEP 3 — IMPORT IN YOUR DART FILE

```
import 'package:http/http.dart' as http;

// The 'as http' part lets us write
// http.get(...) and http.post(...)
// instead of importing each function
// separately.
```

Don't forget to enable internet access on Android — add the permission to AndroidManifest.xml.

GET — fetch data

Use http.get() to ask the server for data.

EXAMPLE

```
Future<void> fetchUsers() async {
  final url = Uri.parse(
    'https://jsonplaceholder.typicode.com/users',
  );

  final response = await http.get(url);

  if (response.statusCode == 200) {
    print(response.body); // raw JSON string
  } else {
    print('Failed: ${response.statusCode}');
  }
}
```

WHAT EACH PIECE DOES

`Uri.parse`

Turns the URL string into a Uri

`await`

Waits for the server's reply

`statusCode`

200 means success

`body`

The actual data, as a string

POST — send data

Use http.post() to send data to the server (create a record, log in, etc).

EXAMPLE

```
import 'dart:convert';

Future<void> createUser() async {
  final url = Uri.parse(
    'https://jsonplaceholder.typicode.com/users',
  );

  final response = await http.post(
    url,
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'name': 'Hama',
      'email': 'hama@example.com',
    }),
  );

  print(response.statusCode); // 201 = created
}
```

GET VS POST

GET

Reads data — no body needed

POST

Creates / sends — body has the data

ALSO EXISTS

`http.put, http.delete`

From JSON to Dart object

The response is a JSON string. Convert it into a real Dart object you can use.

MAKE A MODEL CLASS

```
class User {
  final String name;
  final String email;

  User({required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      name: json['name'],
      email: json['email'],
    );
  }
}
```

USE IT IN YOUR FETCH

```
import 'dart:convert';

Future<List<User>> fetchUsers() async {
  final response = await http.get(url);

  // Parse JSON string to a Dart List
  final List data = jsonDecode(response.body);

  // Convert each item to a User
  return data
    .map((json) => User.fromJson(json))
    .toList();
}
```

FutureBuilder — show fetched data

FutureBuilder waits for an async operation and rebuilds the UI when the data arrives.

USE IT IN BUILD()

```
FutureBuilder<List<User>>(
  future: fetchUsers(),
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      final users = snapshot.data!;
      return ListView(
        children: users.map(
          (u) => ListTile(title: Text(u.name)),
        ).toList(),
      );
    }
    return CircularProgressIndicator();
  },
)
```

THE SNAPSHOT

Loading

Waiting for response

Has data

Show the result

Has error

Show an error message

Handling all three states

A good API screen shows something useful while loading, on success, and on failure.

```
FutureBuilder<List<User>>(
  future: fetchUsers(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return Center(child: CircularProgressIndicator());
    }

    if (snapshot.hasError) {
      return Center(child: Text('Something went wrong'));
    }

    final users = snapshot.data ?? [];
    return ListView(
      children: users.map((u) => ListTile(title: Text(u.name))).toList(),
    );
  },
)
```

EXERCISE · TRY IT NOW

Fetch and display real users

Build a screen that fetches users from a public API and shows their names.

YOUR TASK

1. Add the http package to pubspec.yaml
2. Create a User model with fromJson
3. Write fetchUsers() — call the API
4. Use FutureBuilder to show the list
5. Show a loading spinner while waiting

USE THIS PUBLIC API

```
https://jsonplaceholder.typicode.com/users
```

EXPECTED RESULT

WRAP UP

What we covered today

YOU NOW KNOW

- ✓ What an API is — request & response
- ✓ The http package: get, post, put, delete
- ✓ JSON to Dart with fromJson factories
- ✓ FutureBuilder to display async data
- ✓ Loading and error states in the UI

THANK YOU

Questions?